```
import numpy, pandas, matplotlib, seaborn
import scipy, sklearn, mlxtend
print("All required packages are installed.")
# Create the Dataset
import pandas as pd
data = {
 "Student_ID": ["S%02d" % i for i in range(1, 16)],
 "Study_Hours": [2,3,1,4,5,2,6,1,3,4,2,5,1,3,6],
 "Attendance": [65,70,55,80,85,60,90,50,75,82,68,88,52,72,92],
 "Internal_Mark": [48,55,40,65,72,45,80,35,58,68,50,75,38,56,82],
 "Assignment_Mark": [50,58,42,68,75,48,82,38,60,70,52,78,40,59,85],
 "Final_Mark": [52,60,45,70,78,50,85,40,63,73,55,80,42,61,88],
 "Result": ["Pass","Pass","Fail","Pass","Pass","Pass","Pass","Fail",
            "Pass","Pass","Pass","Pass","Fail","Pass","Pass"]}
pd.DataFrame(data).to_csv("student_performance.csv", index=False)
print("Dataset created")
# NumPy: Statistics of the Final Marks
import numpy as np
marks = np.array([52,60,45,70,78,50,85,40,63,73,55,80,42,61,88])
print("Mean:", np.mean(marks))
print("Median:", np.median(marks))
print("Minimum:", np.min(marks))
print("Maximum:", np.max(marks))
print("Standard deviation:",np.std(marks))
# NumPy: Working on the Whole Array
print("Marks + 5 grace:", marks + 5)
print("Above 70:", marks[marks > 70])
print("Count above 70:", np.sum(marks > 70))
print("Sorted:", np.sort(marks))
# Pandas: Load and Look at the Data
df = pd.read_csv("student_performance.csv")
print(df.head())
print(df.tail())
print("Shape:", df.shape)
# Pandas: Statistical Summary
print(df.describe())
```

```
# Pandas: Missing Values and Duplicates

print(df.isnull().sum())

print("Duplicate rows:", df.duplicated().sum())

# Pandas: Filter and Group

high = df[df["Final_Mark"] >= 70]

print(high)

print(df.groupby("Result")["Final_Mark"].mean())

# Pictures of the data

# Matplotlib: Histogram

import matplotlib.pyplot as plt

plt.hist(df["Final_Mark"], bins=5, edgecolor="black")

plt.xlabel("Final Mark")

plt.ylabel("Number of Students")

plt.title("Distribution of Final Marks")

plt.show()

# Matplotlib: Scatter Plot

plt.scatter(df["Study_Hours"], df["Final_Mark"])

plt.xlabel("Study Hours")

plt.ylabel("Final Mark")

plt.title("Study Hours vs Final Mark")

plt.grid(True)

plt.show()

# Seaborn: Box Plot

import seaborn as sns

plt.boxplot(df["Final_Mark"])

plt.title("Final Mark by Result")

plt.show()

# Seaborn: Correlation Heatmap

# corr computes the correlation of every pair of numeric columns. heatmap colours the
table.

num = df.drop(columns=["Result","Student_ID"])

sns.heatmap(num.corr(), annot=True)

plt.title("Correlation Matrix")

plt.show()

# SciPy, Scikit-learn and Mlxtend- Statistics, learning and rules

# SciPy: Correlation and z-Scores

from scipy import stats
```

```
r, p = stats.pearsonr(df["Study_Hours"], df["Final_Mark"])
print("r =", r, " p-value =",p)

# Scikit-learn: Split the Data
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier as DTC
from sklearn.metrics import accuracy_score as acc, confusion_matrix
,f1_score,recall_score
y = df["Result"]
X = df.drop(columns=["Result","Student_ID"])
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.20)
model = DTC()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", acc(y_test, y_pred)*100)
print(confusion_matrix(y_test, y_pred))

# Mlxtend: Frequent Itemsets
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
baskets = [["Milk","Bread"], ["Milk","Bread","Butter"],
           ["Bread","Butter"], ["Milk","Bread","Butter"],
           ["Milk","Butter"]]
te = TransactionEncoder()
onehot = pd.DataFrame(te.fit_transform(baskets), columns=te.columns_)
print(apriori(onehot, min_support=0.6, use_colnames=True))
```

```
import pandas as pd
import matplotlib.pyplot as plt
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
transactions = [
    ["Milk", "Egg", "Bread", "Butter"],
    ["Milk", "Butter", "Egg", "ketchup"],
    ["Bread", "Butter", "ketchup"],
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter", "Cookies"],
    ["Milk", "Bread", "Butter", "Cookies"],
    ["Milk", "Cookies"],
    ["Milk", "Bread", "Butter"],
    ["Bread", "Butter", "Egg", "Cookies"],
    ["Milk", "Butter", "Bread"],
    ["Milk", "Bread", "Butter"],
    ["Milk", "Bread", "Cookie", "ketchup"]
]
m=0
for n in transactions:
    m+=1
    print(m,n)
# One-hot encoding
te = TransactionEncoder()
tr=te.fit_transform(transactions)
onehot = pd.DataFrame(tr,columns=te.columns_)
print(onehot.astype(int))

# Count items
counts = onehot.sum()
print(counts)
```

```
counts.plot(kind="bar")
# plt.axhline(0.4 * len(onehot), color="red", linestyle="--")
plt.ylabel("Number of transactions")
plt.title("Item Counts")
plt.show()
# Association rules (Lift >= 1)
freq = apriori(onehot, min_support=0.4, use_colnames=True)
rules = association_rules(freq, metric="lift", min_threshold=1)
show = rules[["antecedents", "consequents", "support",
              "confidence", "lift"]]
for i in ["antecedents", "consequents"]:
    l=[]
    for j in show[i]:
      l.append(",".join(list(j)))
   show[i]=l
print(show)
# Association rules (Confidence >= 0.75)
r2 = association_rules(freq, metric="confidence",
                       min_threshold=0.75)
show = r2[["antecedents", "consequents", "support",
           "confidence", "lift"]]
for i in ["antecedents", "consequents"]:
   l=[]
   for j in show[i]:
      l.append(",".join(list(j)))
   show[i]=l
print(show)
```

10. **Implementation of Hierarchical and DBSCAN Clustering**

```
pip install numpy pandas matplotlib scipy scikit-learn
```

Enter the Data Points

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([(5,19), (4,17), (11,25), (6,22), (12,21),
        (10,21), (14,24), (3,16), (10,24), (4,21)])
for i, p in enumerate(X):
   print("Point", i, "=", tuple(int(v) for v in p))
```

Build the Merge Table

```
from scipy.cluster.hierarchy import linkage, dendrogram
Z = linkage(X, method="average")
print(" merged      distance  size")
for a, b, dist, size in Z:
   print("%3d and %-3d  %7.2f   %d" % (a, b, dist, size))
```

Draw the Dendrogram

```
plt.figure(figsize=(8, 4.5))
dendrogram(Z)
plt.title("Dendrogram (average linkage)")
plt.xlabel("Point number"); plt.ylabel("Distance")
plt.show()
```

Form Two Clusters

```
from sklearn.cluster import AgglomerativeClustering
model = AgglomerativeClustering(n_clusters=2, linkage="average")
labels = model.fit_predict(X)
print("Cluster labels:", labels)
for c in [0, 1]:
   print("Cluster", c, ":", [tuple(int(v) for v in p)
                  for p in X[labels == c]])
```

Plot the Clusters

```
plt.figure(figsize=(7, 4.5))
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="rainbow", s=90)
for i, p in enumerate(X):
    plt.annotate(str(i), (p[0] + 0.15, p[1] + 0.15))
plt.xlabel("x"); plt.ylabel("y")
plt.title("Agglomerative Clustering (2 clusters)")
plt.grid(True)
plt.show()
# Part B – DBSCAN
from sklearn.datasets import make_blobs
B, _ = make_blobs(n_samples=750, n_features=2, centers=3,
                  cluster_std=0.4, random_state=0)
print("Shape:", B.shape)
print(B[:3].round(2))
# Run DBSCAN
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=4).fit(B)
lab = dbscan.labels_
n_clusters = len(set(lab)) - (1 if -1 in lab else 0)
print("Clusters found:", n_clusters)
print("Noise points:", int((lab == -1).sum()))
print("Cluster sizes:", np.bincount(lab[lab >= 0]))
print("Core points:", len(dbscan.core_sample_indices_))
# Plot the Clusters
plt.figure(figsize=(7, 4.5))
plt.scatter(B[lab >= 0, 0], B[lab >= 0, 1], c=lab[lab >= 0],
            cmap="viridis", s=15)
plt.scatter(B[lab == -1, 0], B[lab == -1, 1], c="black",
            marker="x", s=60, label="Noise")
plt.xlabel("X Axis"); plt.ylabel("Y Axis")
plt.title("DBSCAN (eps = 0.5, min_samples = 4)")
plt.legend()
```

```
plt.show()


# Effect of eps
for e in [0.1, 0.2, 0.3, 0.5, 1.0]:
    l = DBSCAN(eps=e, min_samples=4).fit(B).labels_
    k = len(set(l)) - (1 if -1 in l else 0)
    print("eps =", e, " clusters =", k,
          " noise =", int((l == -1).sum()))
from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
M, _ = make_moons(n_samples=300, noise=0.06, random_state=0)
km = KMeans(n_clusters=2, random_state=0, n_init=10).fit_predict(M)
db = DBSCAN(eps=0.2, min_samples=4).fit_predict(M)
print("DBSCAN clusters:", len(set(db)) - (1 if -1 in db else 0),
      " noise:", int((db == -1).sum()))
# Compare the Two Results
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
ax[0].scatter(M[:, 0], M[:, 1], c=km, cmap="coolwarm", s=15)
ax[0].set_title("K-Means (K = 2)")
ax[1].scatter(M[:, 0], M[:, 1], c=db, cmap="coolwarm", s=15)
ax[1].set_title("DBSCAN (eps = 0.2)")
plt.show()
```